# Checkpoint 1 Project Report

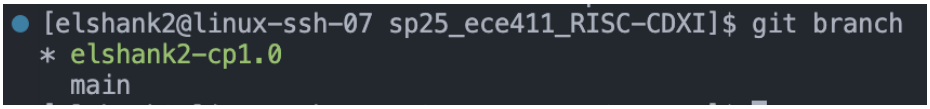
Elizabeth Shank, Tessa Samuels, Sairaj Shetye

**Implemented Features**

- Fetch stage with instruction queue
- Cacheline adapter (arbiter)
- Parameterizable FIFO
- Modified cache - with cacheline buffer for instruction cache

Decided on using explicit register renaming for future checkpoints

**3/25:**

- Added mp_ooo into the RISC-CVXI repo (sairaj)
  - Followed instructions in mp-setup
  - mkdir ece411_group
  - cd ece411_group
  - git init
  - git remote add origin git@github.com:illinois-ece411/sp25_ece411_RISC-CDXI
  - git remote add release git@github.com:illinois-cs-coursework/sp25_ece411_.release.git
  - git pull origin main
  - git branch -m main
  - git fetch release
  - git cherry-pick afca87bfcbc203ffe79bc44ea0881a25e6bbe9dc
  - git push --set-upstream origin main
- How to add to your local dir:
  - mkdir ece411_group
  - cd ece411_group
  - git clone https://github.com/illinois-ece411/sp25_ece411_RISC-CDXI.git
- Now adding branches:
  - In format of netid-cp1.number
    - elshank2-cp1.0
    - elshank2-cp2.0
  - Create branch using github site, then follow instructions below
  - git branch: shows you local branches
  - 

  - git checkout -b new-branch-name: creates a new branch and switches to it
  - git push origin new-branch-name: push new branch to repo

- 
```
[elshank2@linux-ssh-07 sp25_ece411_RISC-CDXI]$ git push origin elshank2-cp1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'elshank2-cp1.0' on GitHub by visiting:
remote:      https://github.com/illinois-ece411/sp25_ece411_RISC-CDXI/pull/new/elshank2-cp1.0
remote:
To https://github.com/illinois-ece411/sp25_ece411_RISC-CDXI.git
 * [new branch]      elshank2-cp1.0 -> elshank2-cp1.0
```

Switching to another branch in Git. Straightforward answer,

97

git-checkout - Switch branches or restore working tree files

```
git fetch origin         # <---- This will fetch the branch
git checkout branch_name # <---  Switching the branch
```

Before switching the branch, make sure you don't have any modified files. In that case, you can commit the changes or you can stash it.

Share Improve this answer Follow

edited Dec 10, 2023 at 1:06 — Peter Mortensen — 31.6k • 22 • 110 • 133

answered Dec 4, 2017 at 11:23 — danglingpointer — 4,930 • 3 • 26 • 45

**3/26:**

- Advanced features: 2 way superscalar, early branch recover would go over into extra credit, choose a prefetcher
- Decided on using ERR for
    - Tomasulo's Algorithm is not perfect, as it has a number of limitations that result in a lower clock frequency, higher area, and higher power consumption. One solution to this is a more modern algorithm: Explicit Register Renaming, and reading data after the issue stage.
- Need 2 separate cache for imem & dmem, don't really need to check address. Just can wait for imem resp and then use it.
- Cache Line adaptor contacts data. Can use counter
- Will want to change clock frequency, higher frequency gives better (less) delay
- 2 line buffers, doubling certain things
    - dont need to double inst queue, just take out 2 inst at once
- Whole group worked on datapath just for CP1, lots of discussion about implementation.

**3/27:**

- Sairaj: queue, linebuffer, fetch
- Elizabeth: arbiter
- Tessa: datapath
- Whole group: combining all our code so far
    - Confusion hooking up line buffer with arbiter/cache line. Hooked up the line buffer straight to the arbiter so there is not a delay.
    - Needed to change instruction cache so it outputs the whole cache line (256 bits) instead of the 32 from mp_cache.
    - PC register was not an actual register, fixed that.
    - Comb loop :(.

Questions

- What exactly goes into the instruction queue? Does PC & order go in with instruction? If not, where do they go?

**3/28:**

- Sairaj & Elizabeth realized our cache line needed to be in the cache and not its own module, so modified code. Comb loop still remains
- With our cpu testbench realized the combinational loop caused an infinite loop
    - One attempted quick fix of the comb loop was to set pc-next using always_ff instead of always_comb, however this resulted in incorrect PC behavior
- Comb loop solution:

```
// I use ufp_addr directly for tag and set since tag/set are a clock cycle late in register
if (lb_data_reg != '0 && lb_addr_reg != '0 && lb_tag == ufp_addr_reg[31:9] && lb_set == ufp_addr_reg[8:5]) begin
    ufp_rdata = lb_data_reg;
```

    - Had to use ufp_addr_reg and reset it to 0xAAAAA000
- There is a one cycle delay when reading directly from memory since our code used a register.

# CP1 Tests:

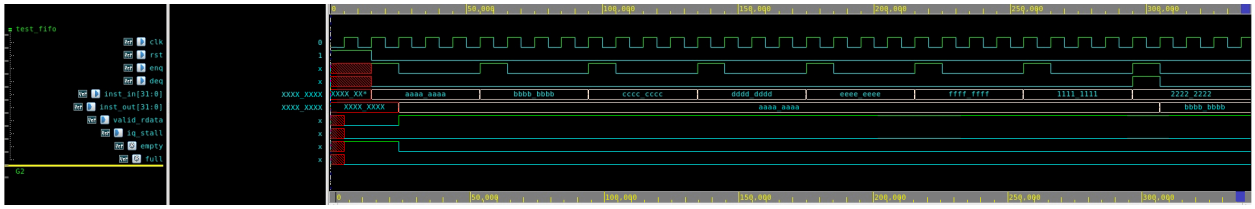
**Queue (FIFO):**

almost_empty_tb.sv:



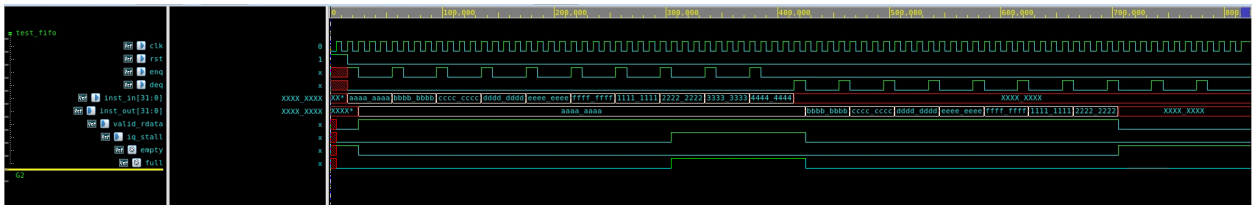
Enqueues a singular instruction to the FIFO, then attempts to enqueue and dequeue at the same time. The empty signal correctly stays low throughout the entire process.

almost_full_tb.sv:



Fills the FIFO with 7 instructions, 1 instruction away from being full. Similar to the prior waveform, enqueue and dequeue are then done simultaneously with the full signal staying low throughout the entire process.

full_empty_tb.sv:



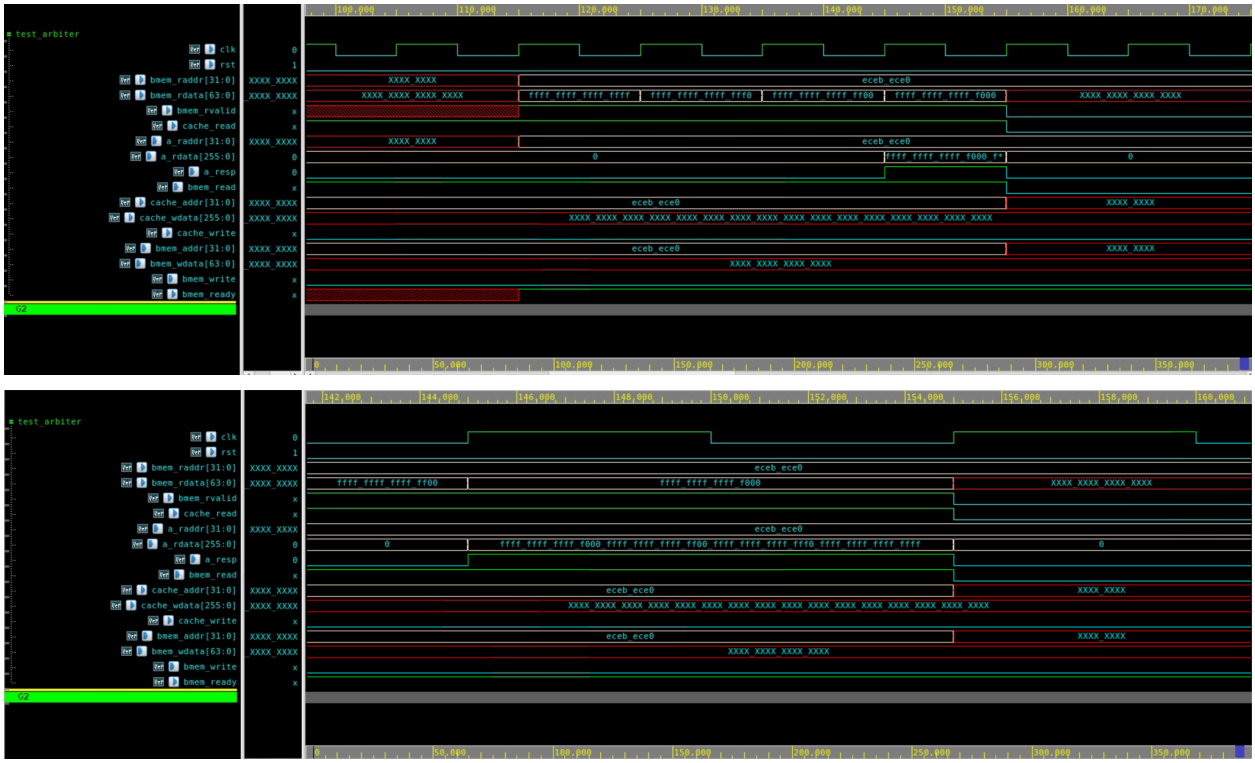
Fills the FIFO incrementally, attempting to enqueue 10 instructions into the queue. The last two instructions get blocked, since the depth of the FIFO is 8 and iq_stall becomes raised high. We then read the values in the FIFO through inst_out as we attempt to dequeue 10 times. The last 2 dequeues should not go through and should not move the head ptr, since the FIFO is already empty.

**Arbiter (Read + Write):**

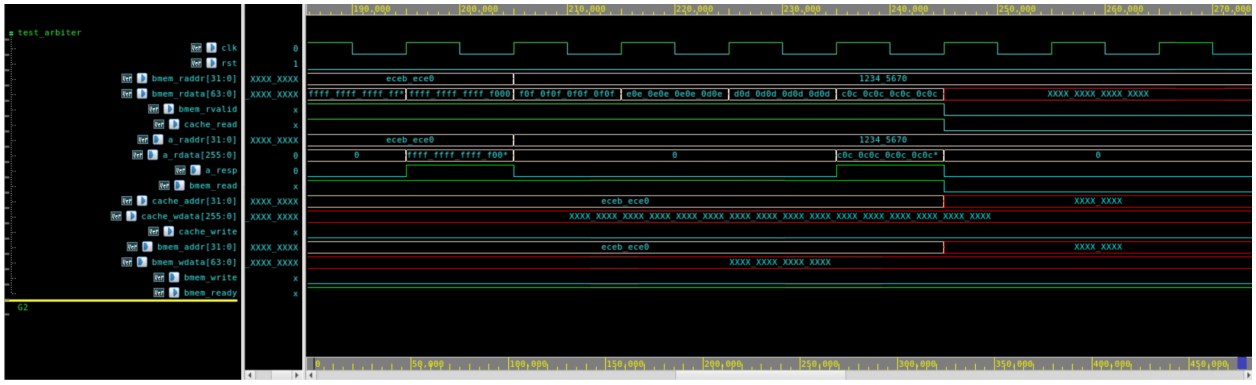
Read: takes 4 64 bit chunks from bmem_rdata and turns into 256 bit and sends dpf_resp (a_resp in waveform) to cache high

Input: bram_rdata - 64

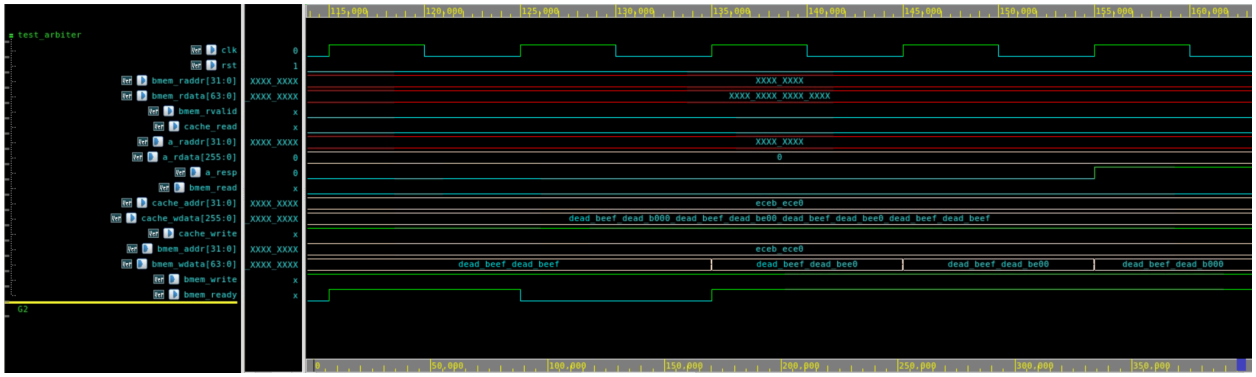
Output: a_rdata (to cache) - 256



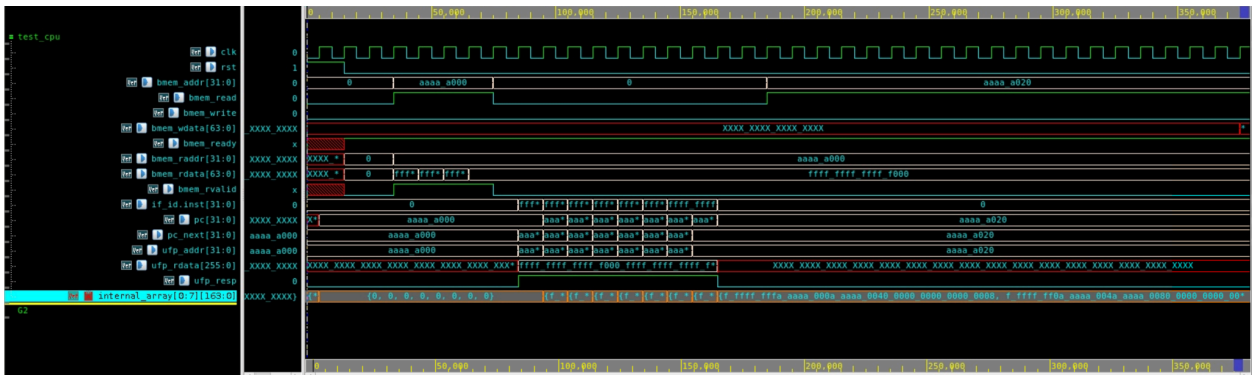
Version with two different values



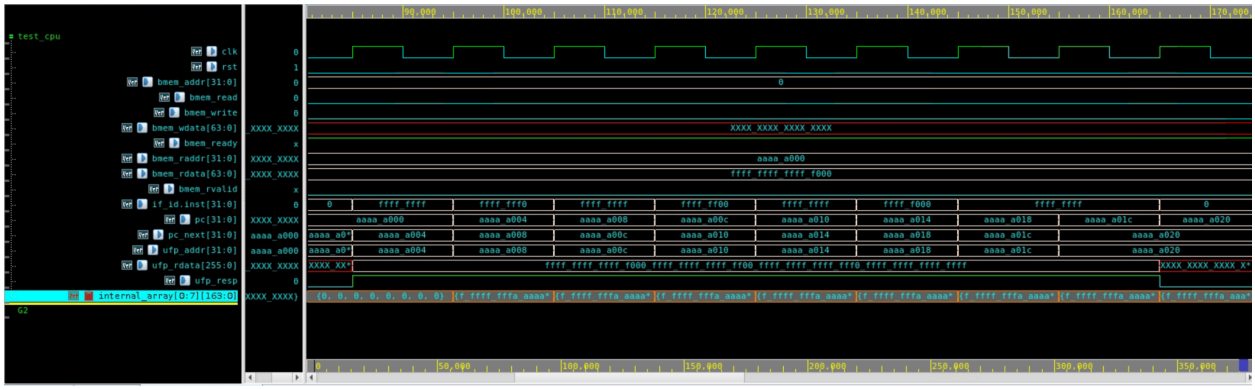
Write: takes 256 bit when cache sends write request and breaks apart into 4 64 bit chunks to bmem_wdata
Input: cache_wdata - 256
Output: bmem_wdata - 64
Shows that does not move onto next 64 bit chunk until bram is ready, sets a_resp (dfp_resp) high at end to send to cache and show end of write



**Full CPU:**
Shows PC, PC_next, instructions, reading from cacheline buffer, going into instruction queue



Internal_array: instruction queue, fills up as PC increments and more instructions are enqueued, causes PC to stall once the queue is filled up
Testbench acts as memory and performs a bmem_read when CPU cache asks for cacheline at address xAAAAA000


test_cpu
clk
rst
bmem_addr[31:0]
bmem_read
bmem_write
bmem_wdata[63:0]
bmem_ready
bmem_raddr[31:0]
bmem_rdata[63:0]
bmem_rvalid
if_id.inst[31:0]
pc[31:0]
pc_next[31:0]
ufp_addr[31:0]
ufp_rdata[255:0]
ufp_resp
internal_array[0:7][163:0]
G2
90,000
100,000
110,000
120,000
130,000
140,000
150,000
160,000
170,000
0
50,000
100,000
150,000
200,000
250,000
300,000
350,000